Generative Compression for Differentiable Opaque Meshes

Technical report

Wenhao Su

November 2, 2026

Abstract

MeshSplatting produces an opaque, engine-ready triangle mesh with per-vertex spherical harmonics (SH), but its FP32 output is 430–740 MB per scene. We keep MeshSplatting's optimisation untouched and add two training-time decisions and one delivery codec: triangles are split only when the distortion they remove is worth the bytes they add, the same price is applied in reverse as an edge-collapse prune, and the trained mesh is encoded into a single file whose per-vertex SH is replaced by shared dictionary indices. One rate–distortion parameter λ traces the whole curve. On Mip-NeRF 360 the 642 MB FP32 mesh becomes 26.6 MB at -0.24 dB ($24\times$), 9.0 MB at -0.92 dB ($72\times$) and 4.3 MB at -1.68 dB ($151\times$); the delivered asset is a standard mesh that decodes in 2–6 s on one CPU core and renders at 145–350 fps on a phone.

1 Methodology

We keep MeshSplatting's optimisation untouched and change two things: *which* triangles get split during growth (priced in bytes), and a reverse prune under the same price. The trained mesh then goes through a codec that replaces per-vertex SH by shared dictionary indices. The vertex count is no longer a hand-set cap but an output of one rate–distortion trade-off λ. Figure 1 shows where the pieces sit.

1.1 Where we intervene in MeshSplatting

Upstream training has two stages. **Stage 1 (iterations 0–10 000)** grows a soft triangle soup: every 500 iterations a fixed fraction of triangles (`add_percentage`) is split, chosen by importance sampling, up to a hand-set cap. At iteration 11 000 a restricted Delaunay rebuild re-triangulates the soup. **Stage 2 (11 000–30 000)** optimises the now-opaque mesh (opacity schedule, $2\times/4\times$ supersampling after 20k/25k).

Our changes sit only in the growth decisions:

1. **Stage 1, the split rule.** The importance-sampled split is replaced by a priced one: a triangle is split only if the distortion it would remove is worth the bytes it adds (Section 1.2). Growth stops by itself when no triangle passes.
2. **Stage 2, λ-prune rounds** at iterations 12k / 14k / 16k / 18k (after the Delaunay rebuild, before supersampling): edges whose faces carry too little residual for the bytes they cost are collapsed, at most 15 % of the vertices per round, with a fold-over guard. Same λ, applied in reverse.

Losses, learning rates, opacity schedule, Delaunay rebuild, supersampling and final cleaning are upstream code imported as is. The appearance optimisation is MeshSplatting's own.

1.2 How the rate cost is estimated and used

For every triangle t we accumulate during each densification interval its **distortion mass** ΔD_t: the per-pixel L1 error summed over the pixels whose front-most triangle is t (from the rasteriser's per-pixel triangle ids), averaged per view and normalised per megapixel. Its **rate** ΔR_t is the number

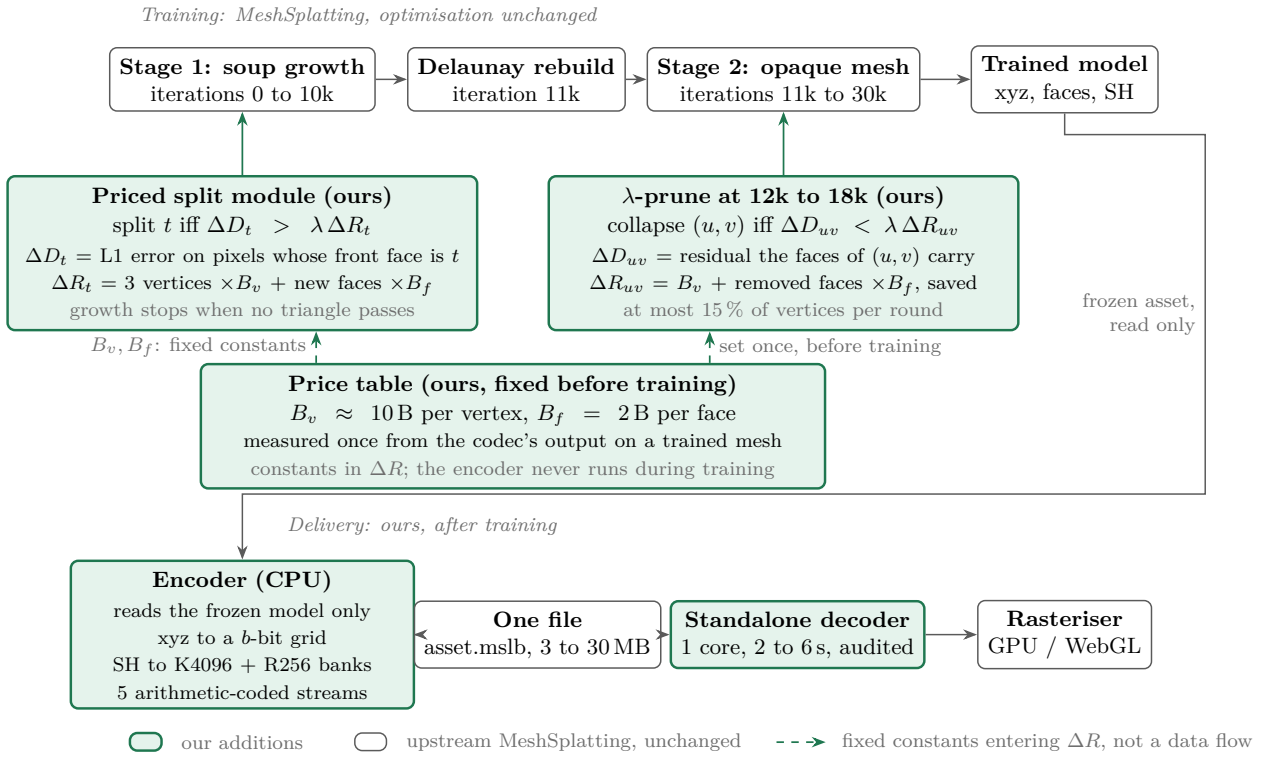


Figure 1: Pipeline: two priced decisions during training, one dictionary codec at delivery. Grey boxes are MeshSplatting as published; tinted boxes are ours. The price table is a pair of constants measured once from the codec's output; the encoder itself runs only on the frozen, finished model and cannot influence how the mesh grew.

of bytes a split would add to the *delivered file*: three new vertices times the bytes per vertex our codec actually spends ($B_v \approx 10$ B at the scene's position precision: connectivity + position residual + SH indices + DC), plus $B_f = 2$ B per new face.

$$\text{split } t \iff \frac{\Delta D_t}{\Delta R_t} > \lambda, \qquad \text{collapse } (u, v) \iff \frac{D_u + D_v - D_{uv}}{\Delta R_{uv}} < \lambda. \tag{1}$$

λ is in units of L1 mass per byte. Four values (6.2×10^{-4}, 1.85×10^{-3}, 6.2×10^{-3}, 1.85×10^{-2}) trace the RD curve; one λ stops growth at 15–100 % of the dense vertex count depending on the scene, i.e. the budget adapts to scene content. For the prune, $D_u + D_v - D_{uv}$ is the residual mass the faces around edge (u, v) carry and ΔR_{uv} the bytes the collapse saves.

The encoder is not in the training loop. Its only contribution to training is the price table: B_v and B_f, measured once from the codec's output on a trained mesh and then held constant for every run. ΔR_t is computed from these two constants, never by encoding anything during training. The encoder itself runs once, after iteration 30k, on the frozen model, and cannot influence how the mesh grew.

1.3 Encoder and decoder

The encoder takes the finished mesh (positions, faces, 48 SH coefficients per vertex, one opacity scalar) and writes one file.

- **Positions** are snapped to a b-bit grid with $b = \lceil \log_2(\text{extent}/(0.1 \cdot \text{median edge})) \rceil$ (14–18 bits per scene), reordered with meshoptimizer for locality, and coded as causal-neighbour residuals.
- **Connectivity** is coded with a cached-edge arithmetic coder after the same reorder; it is the largest stream and is kept lossless.
- **SH** is the lever: the 45 non-DC coefficients of every vertex are replaced by an index into a main dictionary of 4096 vectors plus a residual dictionary of 256, both learnt by k-means on the scene

and refit for 400 steps against the rendering loss. Indices are coded with a 1-ring context model; the DC term is quantised to 7 bits and zstd-coded.

- **Banks and header** (the two dictionaries, one opacity scalar, grid parameters) are a fixed 0.6 MB.

The decoder is a standalone program with no access to the training data: it reads the five streams, rebuilds positions, faces and per-vertex SH, and hands a standard mesh to any rasteriser. It runs in 2–6 s on one CPU core. For a Room asset at full growth the file is roughly 16 MB geometry + 15 MB SH; the SH dictionary is what makes a 740 MB FP32 model fit in 31 MB at −0.4 dB before any growth control.

2 Experiments

Setup. Mip-NeRF 360 (9 scenes) and Tanks & Temples (truck, train), test split = every 8th view. FP32 = our reproduction of MeshSplatting with the upstream flags (`-indoor` on room, counter, kitchen, bonsai); it matches the paper within ±0.1 dB. MB = 10^6 bytes of the delivered file; ratio = FP32 size / delivered size. Each λ point is one training run plus one encode; nothing is tuned per scene. LPIPS in our delivered-asset rows is the pipeline convention (inputs in $[-1, 1]$), about 0.03 higher than the 3DGS convention used by the paper and by every published baseline; the unified evaluation pass will put all rows on the 3DGS convention.

2.1 Headline: the RD curve

Table 1: Dataset means. Mip-NeRF 360, 9 scenes, unless marked T&T.

point	MB	ratio	PSNR / SSIM / LPIPS	ΔPSNR vs FP32
FP32 (our reproduction)	642	1×	24.81 / 0.731 / 0.309	–
$\lambda = 6.2 \times 10^{-4}$	26.6	24×	24.56 / 0.717 / 0.385	−0.24
$\lambda = 1.85 \times 10^{-3}$	20.0	32×	24.44 / 0.709 / 0.405	−0.37
$\lambda = 6.2 \times 10^{-3}$	9.0	72×	23.89 / 0.672 / 0.464	−0.92
$\lambda = 1.85 \times 10^{-2}$	4.3	151×	23.12 / 0.624 / 0.518	−1.68
T&T FP32	459	1×	20.64 / 0.760 / 0.272	–
T&T $\lambda = 6.2 \times 10^{-3}$	11.4	40×	20.52 / 0.737 / 0.358	−0.12

2.2 Per scene

Table 2: Per scene: FP32 model and the four λ points. Each cell is MB (ratio) · PSNR.

scene	FP32 MB	FP32 PSNR	λ 6.2e−4	λ 1.85e−3	λ 6.2e−3	λ 1.85e−2
room	597	28.48	22.1 (27×) · 28.25	12.4 (48×) · 28.02	5.4 (110×) · 27.30	2.9 (207×) · 26.38
counter	520	26.48	20.7 (25×) · 25.95	13.1 (40×) · 25.84	6.4 (82×) · 25.32	3.3 (155×) · 24.54
kitchen	507	27.46	24.5 (21×) · 27.09	16.5 (31×) · 26.89	8.0 (63×) · 25.97	3.8 (134×) · 24.62
bonsai	678	28.28	24.3 (28×) · 27.89	13.2 (52×) · 27.41	6.1 (112×) · 26.36	3.5 (192×) · 25.36
garden	740	24.75	36.4 (20×) · 24.73	24.6 (30×) · 24.50	10.2 (73×) · 23.65	5.0 (149×) · 22.78
bicycle	680	23.04	31.6 (22×) · 22.99	27.0 (25×) · 23.02	11.9 (57×) · 22.69	5.2 (131×) · 22.08
stump	698	24.99	27.6 (25×) · 24.81	21.4 (33×) · 24.71	8.9 (79×) · 24.12	3.9 (178×) · 23.21
flowers	668	19.45	25.7 (26×) · 19.02	24.2 (28×) · 19.07	12.0 (56×) · 19.21	5.6 (118×) · 18.84
treehill	692	20.39	27.0 (26×) · 20.31	27.4 (25×) · 20.45	11.9 (58×) · 20.38	5.1 (136×) · 20.30
truck	432	22.39	27.3 (16×) · 21.78	19.3 (22×) · 22.20	9.4 (46×) · 22.37	4.4 (98×) · 21.64
train	485	18.90	30.1 (16×) · 18.55	25.4 (19×) · 18.66	13.4 (36×) · 18.68	6.4 (76×) · 18.55
Mip-360 mean (9)	642	24.81	26.6 (24×) · 24.56	20.0 (32×) · 24.44	9.0 (72×) · 23.89	4.3 (151×) · 23.12
T&T mean (2)	459	20.64	28.7 (16×) · 20.17	22.3 (21×) · 20.43	11.4 (40×) · 20.52	5.4 (85×) · 20.10

The same λ lands at 2.9–6.4 MB indoors and 5–13 MB outdoors: the byte budget is decided by scene content, not by a cap. On the hard outdoor scenes (flowers, treehill, truck, train) the two

smallest-λ points are dominated: growth that the price lets through does not buy test PSNR there, which is exactly what the in-the-loop ablation in Section 2.5 measures.

2.3 Same asset, different codecs

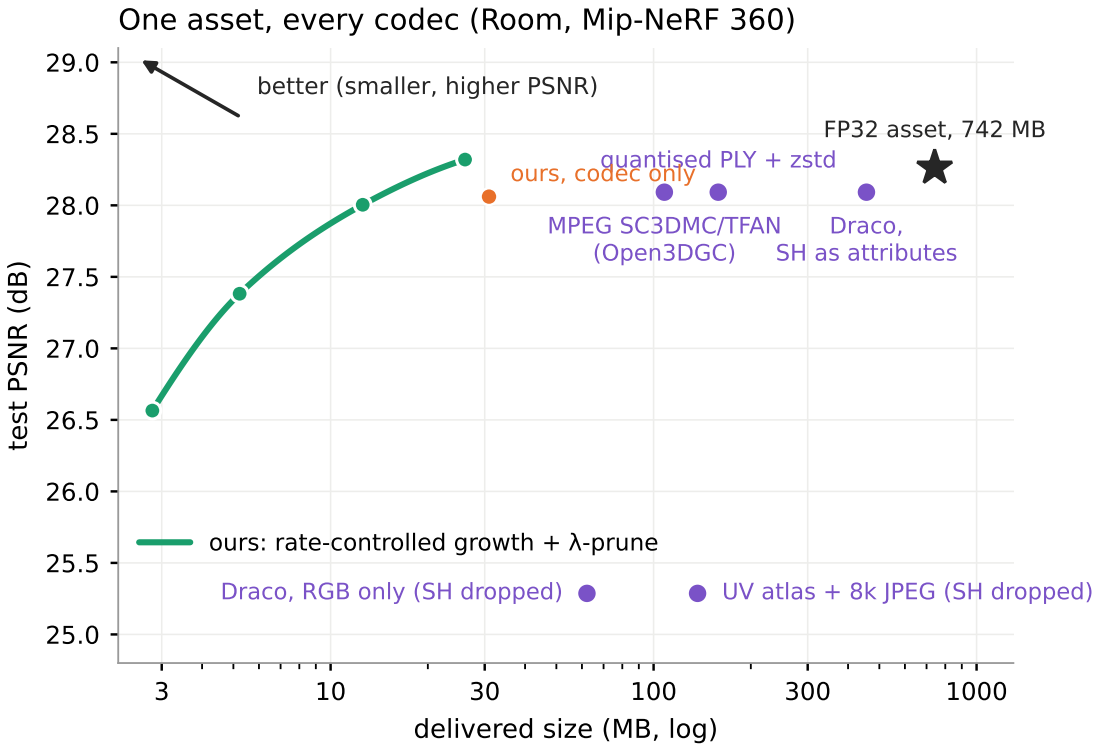


Figure 2: One FP32 Room asset (3.19 M vertices, 742 MB) through generic codecs and through ours. Every point is the same trained model; only the codec differs.

Table 3: Room, same FP32 asset, different routes.

route	MB	PSNR	note
PLY + zstd	158.5	28.09	lossless after 8-bit SH quantisation
MPEG SC3DMC/TFAN (Open3DGC), SH as 8-bit attributes	108.0	≈28.09	strongest generic row; geometry 19.9 + SH 88.1
Draco, SH as attributes	456.1	28.09	splits vertices 4.5×
MPEG or Draco geometry + JPEG texture (4k / 8k atlas)	91–146	25.3	SH dropped, −3.0 dB
ours, codec only (level 1.0, no growth control)	30.9	28.06	3.5× smaller than MPEG
ours, $\lambda = 2 \times 10^{-3}$	13.7	27.99	7.9× smaller than MPEG

Of the 108 → 31 MB gain, about 73 MB comes from replacing per-vertex SH by dictionary indices and about 4 MB from our geometry streams; swapping only the geometry encoder for MPEG costs 13 %. Figure 2 uses the 10/24 Room asset (trained without `-indoor`); every row in it is on that one asset, so it is internally consistent. Garden behaves the same way (MPEG 126 MB vs ours 32.5 MB at equal quality).

2.4 Against other methods

Inside the engine-ready mesh class we reach MiLo's quality at 22× fewer bytes, RaDe-GS's at 175×, GOF's at 352× with +2.3 dB; the authors' own RGB-only PLY (100 MB) is 4× larger than our best point and 0.22 dB higher only because it drops SH entirely. Against 3DGS-class codecs the 3 dB absolute gap is the opaque-mesh representation gap (MeshSplatting is already 2.7 dB below 3DGS uncompressed), not a codec gap; the comparable quantity is the quality retained at equal bytes

Table 4: Cross-method comparison, dataset means. Published numbers verbatim; FPS as reported, hardware differs by row; "–" = not reported. † pipeline LPIPS convention, ≈ 0.03 higher than the 3DGS convention of the other rows.

class	method	Mip-360 PSNR / SSIM / LPIPS	Mip-360 MB	T&T PSNR / SSIM / LPIPS	T&T MB	FPS	mesh
mesh	MeshSplatting, paper (2025)	24.78 / 0.728 / 0.310	100[a]	20.52 / 0.745 / 0.287	–	220 / 190 (M4)	yes
mesh	MeshSplatting FP32, our reproduction	24.81 / 0.731 / 0.309	642	20.64 / 0.760 / 0.272	459	–	yes
mesh	MiLo (SIGGRAPH Asia 2025)	24.09 / 0.688 / 0.323	253	21.46 / 0.706 / 0.348	–	170 / 160 (M4)	yes
mesh	Binary Opacity Grids (SIGGRAPH 2024)	25.62 / – / –	913	–	–	927 (desktop)	yes
mesh	RaDe-GS mesh (2024)	23.56 / 0.668 / 0.361	1100	20.51 / 0.659 / 0.344	–	OOM on M4	yes
mesh	GOF mesh (2024)	20.78 / 0.573 / 0.465	1500	21.69 / 0.690 / 0.326	–	OOM on M4	yes
mesh	**ours,** $\lambda = 6.2 \times 10^{-4}$	24.56 / 0.717 / 0.385†	26.6	20.17 / 0.737 / 0.332†	28.7	350 phone / 820 laptop	yes[b]
mesh	**ours,** $\lambda = 1.85 \times 10^{-3}$	24.44 / 0.709 / 0.405†	20.0	20.43 / 0.745 / 0.331†	22.3	350 phone / 820 laptop	yes[b]
mesh	**ours,** $\lambda = 6.2 \times 10^{-3}$	23.89 / 0.672 / 0.464†	9.0	20.52 / 0.737 / 0.358†	11.4	350 phone / 820 laptop	yes[b]
3DGS	3DGS (SIGGRAPH 2023)	27.46 / 0.812 / 0.222	750.9	23.69 / 0.844 / 0.178	431.0	99 (desktop)	no
3DGS	Scaffold-GS (CVPR 2024)	27.50 / 0.806 / 0.252	253.9	23.96 / 0.853 / 0.177	86.5	135	no
3DGS	HAC++ low (2025)	27.60 / 0.803 / 0.253	8.34	24.22 / 0.849 / 0.190	5.18	130–151	no
3DGS	HAC++ high (2025)	27.82 / 0.811 / 0.231	18.48	24.32 / 0.854 / 0.178	8.63	130–151	no
3DGS	RDO-Gaussian (ECCV 2024)	27.05 / 0.802 / 0.239	23.46	23.34 / 0.835 / 0.195	12.03	–	no
3DGS	Compact3DGS (CVPR 2024)	27.08 / 0.798 / 0.247	48.80	23.32 / 0.831 / 0.201	39.43	–	no
3DGS	LightGaussian (NeurIPS 2024)	27.00 / 0.799 / 0.249	44.54	22.83 / 0.822 / 0.242	22.43	–	no

[a] RGB-only PLY, SH dropped. [b] plus a CPU decode of 2–6 s. Our FPS measured on Room (iPhone 17 Pro / MacBook Pro M2 Max, WebGL2).

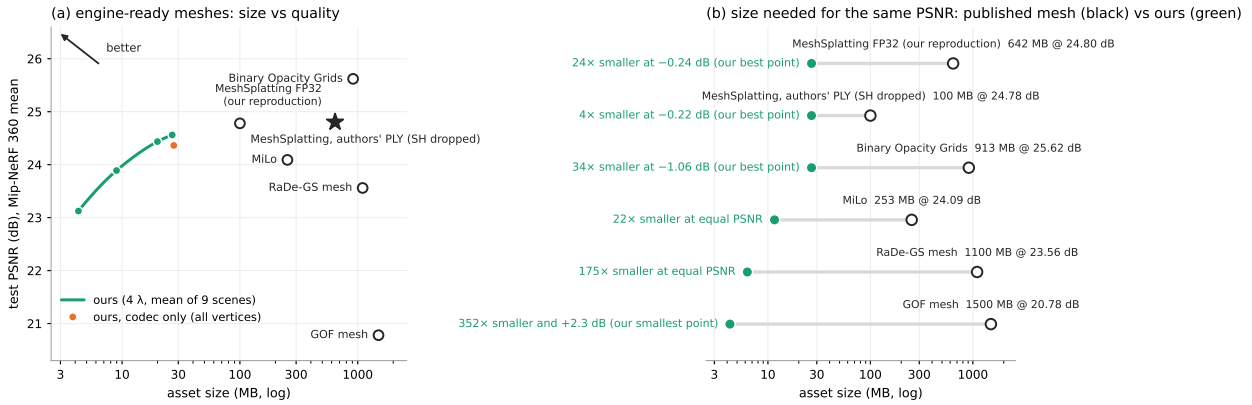


Figure 3: Engine-ready mesh methods, Mip-NeRF 360 means. (a) size vs PSNR; (b) bytes needed to reach each method's PSNR.

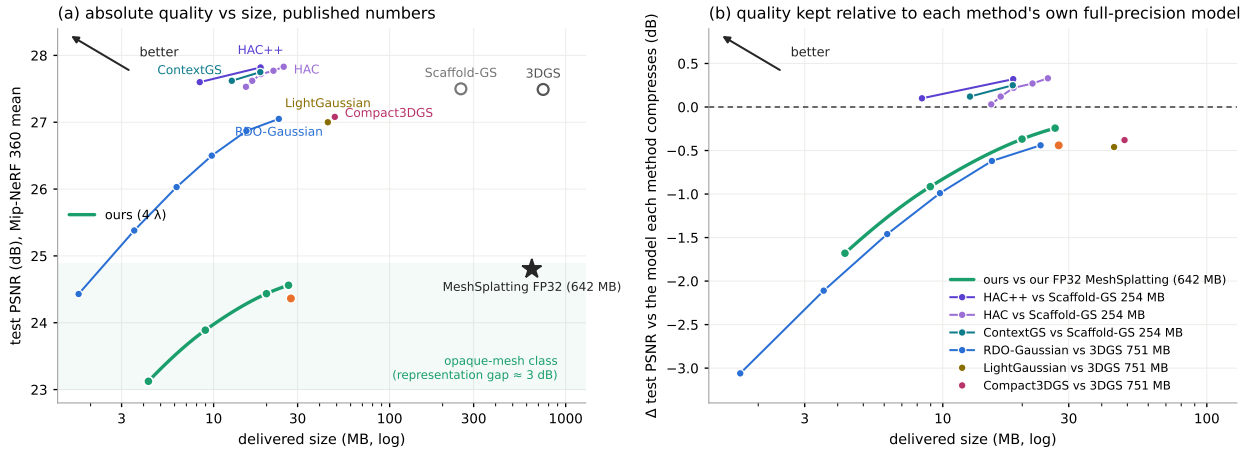


Figure 4: 3DGS-class codecs. (a) absolute numbers; (b) quality retained relative to each method's own full-precision model.

(Figure 4b), where we retain more than RDO-Gaussian, and the HAC family's $+0.1 \ldots + 0.3$ dB comes from Scaffold-GS being more compressible to start with.

2.5 Ablation: is the rate in the loop worth it?

With the codec held fixed, post-hoc edge collapse to 10 MB falls to 22.7 dB mean, training under a fixed vertex cap reaches 24.9 dB, and λ-priced growth plus prune reaches 25.0 dB and keeps going down to 4 MB. At matched vertex count the in-the-loop choice wins on every scene, by $+0.3$ to $+4.3$ dB. The prune rounds alone are a small effect ($-8\,\%$ bytes at $+0.01$–0.02 dB on Room); the gain is in where growth is allowed to happen.

2.6 Deployability

Table 5: Deployability at the $\lambda = 6.2 \times 10^{-3}$ point. Laptop = MacBook Pro M2 Max (Chrome), phone = iPhone 17 Pro (Safari), WebGL2 viewer, SH degree 3, full capture resolution.

scene	asset MB	ratio	train min, upstream → ours	CPU decode s (1 core)	laptop ms (fps)	phone ms (fps)
room	5.2	142×	51 → 27 (0.52×)	2.3	1.2 (820)	2.8 (352)
garden	10.2	73×	49 → 24 (0.48×)	4.4	1.8 (566)	6.3 (160)
bicycle	11.9	57×	41 → 25 (0.60×)	5.4	1.9 (523)	6.9 (145)
11-scene mean	9.4	73×	0.57×	4.1	–	–

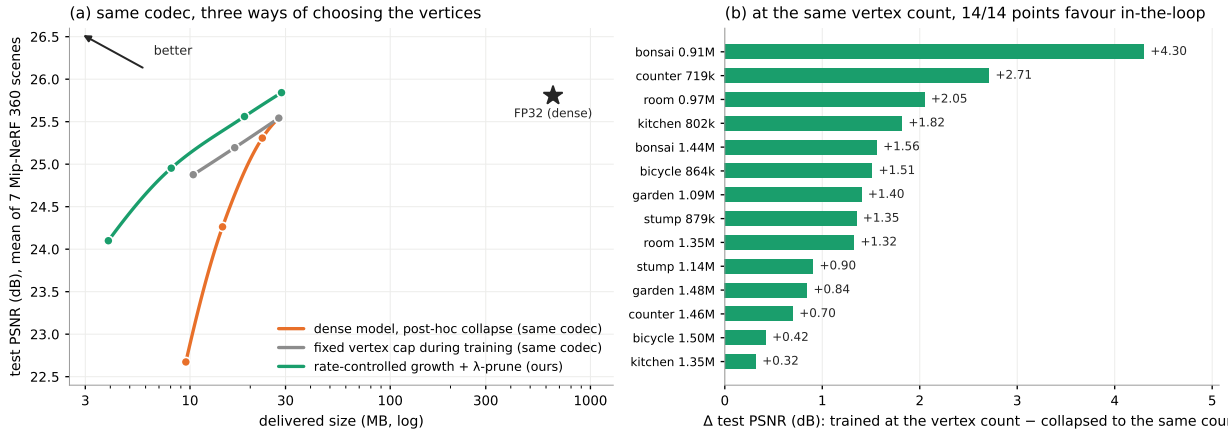


Figure 5: Same codec, three ways of choosing vertices (7 Mip-NeRF 360 scenes, older parameter set): post-hoc edge collapse, fixed vertex cap, λ-growth + prune. (b) At equal vertex count the in-the-loop choice wins on 14 of 14 points.

Training is faster than upstream because the mesh stops growing where the price says so; the decoder is a standalone single-core program with no access to the training data.

2.7 What is still open

- Indoor assets lose 0.5–0.75 dB more to the fixed delivery settings (DC 7-bit, 4096 + 256 dictionary) than outdoor ones; a 0.5 GPU-h re-export diagnostic on room and kitchen is planned.
- Unified evaluation on the 3DGS LPIPS convention ($\approx$ 3 GPU-h) and MPEG rows for the remaining nine scenes (CPU) are pending; neither changes the ordering above.